# INGEGNERIA DEL SOFTWARE

## ESERCITAZIONE 1

Gabriele Paveri-Fontana
Mail: gabriele.paveri@polimi.it

07 Ottobre 2026

# Esercizio 1 *Fraction*

Creare un nuovo *project* e un nuovo *package*.
Al suo interno definire la classe *Fraction*, con attributi e costruttori appropriati, ed implementare i metodi standard ed il metodo *lowestTerms* come da specifica.
Definire poi una classe *Main* per testare la classe *Fraction*.

**Specifiche**:

- Attributi: numeratore e denominatore (interi).
- Costruttori: x/y, x/1, 0/1.
- Metodi standard: *getters&setters, toString, equals (e hashcode)*.
- Metodo *lowestTerms*: riduce la frazione ai minimi termini.

# Esercizio 2 *Polygon*

Definire la classe *Point* e la classe *Polygon*.

**Specifiche**:

- Un punto (*Point*) è identificato dalle sue coordinate nel piano.
- Un poligono (*Polygon*) è un vettore di punti.
- Il numero dei punti che costituiscono un poligono deve essere immutabile.
- Il numero dei punti di un poligono e le loro coordinate devono essere letti da input (si utilizzi lo standard user input *java.util.Scanner*).
- Deve essere possibile ottenere il perimetro di un poligono.
- Metodo *toString()* in *Polygon* con la sintassi [(x1, y1), (x2, y2),…, (x_n, y_n)].

# Esercizio 3a *Set* con vettori

Definire la classe *Set*:
una collezione non ordinata di interi senza duplicati.

**Specifiche**:

- Un insieme (*Set*) è un vettore di interi senza duplicati con una grandezza massima finita e arbitraria (default = 100).
- Deve essere possibile controllare se un elemento è presente in un insieme.
- Deve essere possibile aggiungere e rimuovere elementi da un insieme.
- Deve essere possibile ottenere una rappresentazione testuale di un insieme con la sintassi “{elem1, elem2, …}”.

# Esercizio 3b *Set* con liste

Modificare la classe *Set* appena definita,
utilizzando gli *ArrayList* (*java.util.arraylist*) al posto dei vettori.

**Specifiche**:

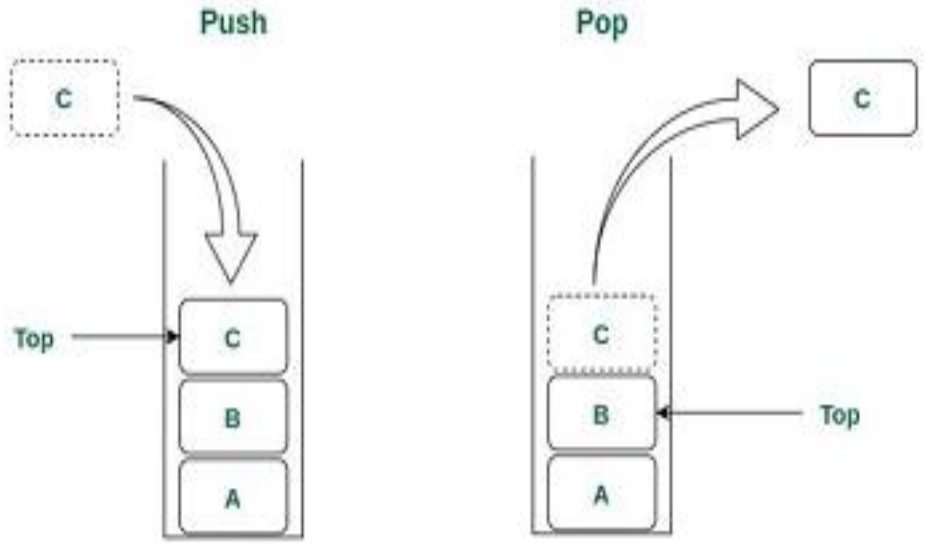
- Un insieme (*Set*) è un *ArrayList* di interi (!!) senza duplicati.
- Tutte le altre specifiche sono le stesse dell'esercizio precedente.

# Esercizio 4 *Stack*

Definire una classe *Stack* di interi:
una struttura dati LIFO (last-in-first-out).

**Specifiche**:

- Uno *Stack* ha una dimensione finita e arbitraria (default = 10, non modificabile).
- Metodo *pop()*: ottiene l'ultimo elemento aggiunto e lo rimuove dallo stack.
- Metodo *push()*: aggiunge un elemento nello stack.



# Esercizio 5 Parameter Evaluation

Illustrare e motivare il valore delle variabili *i*, *counter1* e *counter2* dopo ogni riga del *main*.

```
class Counter {
        int counter = 0;
        public void increment() {
                counter++;
        }
        public void incrementAndSet(int i) {
                i++;
                counter = i;
        }
        public void incrementAndSet(Counter c) {
                c.counter++;
                counter = c.counter
        }
}
```

```
class ParameterEvaluation {
        public static void main(String args[]) {
                Counter counter1 = new Counter();
                counter1.increment();
                int i = 3;
                counter1.incrementAndSet(i);
                Counter counter2 = new Counter();
                counter2.incrementAndSet(i);
                counter1.incrementAndSet(counter2);
        }
}
```

# Esercizio 5 Parameter Evaluation

Illustrare e motivare il valore delle variabili *i*, *counter1* e *counter2* dopo ogni riga del *main*.

```
class ParameterEvaluation {
    public static void main(String args[]) {
        Counter counter1 = new Counter();
        counter1.increment();
        int i = 3;
        counter1.incrementAndSet(i);
        Counter counter2 = new Counter();
        counter2.incrementAndSet(i);
        counter1.incrementAndSet(counter2);
    }
}
```

**Risposta:**

counter1 = 0
counter1 = 1
i = 3, counter1 = 1
i = 3, counter1 = 4
i = 3, counter1 = 4, counter2 = 0
i = 3, counter1 = 4, counter2 = 4
i = 3, counter1 = 5, counter2 = 5

**Tipi primitivi**
passati per *valore*
**Oggetti**
passati per *riferimento*

# Esercizio 6a La classe *String*

Illustrare l'effetto delle istruzioni in rosso sullo heap.
Che uguaglianza c'è tra le quattro *String* ("==" VS "*equals*")?

```
public class StringDemoA {
        public static void main(String args[]) {
                String s1 = "abc";
                String s2 = "abc";
                String s3 = new String("abc");
                String s4 = s3;
        }
}
```

# Esercizio 6a La classe *String*

Illustrare l'effetto delle istruzioni in rosso sullo heap.
Che uguaglianza c'è tra le quattro *String* ("==" VS "*equals*")?

```
public class StringDemoA {
        public static void main(String args[]) {
                String s1 = "abc";
                String s2 = "abc";
                String s3 = new String("abc");
                String s4 = s3;
        }
}
```

**Risposta**:

- s1 e s2 puntano allo stesso oggetto String nello heap ("==")
- s3 ed s4 puntano allo stesso oggetto String nello heap ("==")
- Gli oggetti String puntati da s1 e da s3 sono diversi ("!==")
- *equals* vale *TRUE* per tutte le coppie

  s1.equals(s2)
  s2.equals(s3)
  s3.equals(s4)

# Esercizio 6b *String* VS *StringBuilder*

Cosa stampa il seguente programma?

```
public class StringDemoB {
        public static void main(String[] args){
                String s1 = “Strings”;
                String s2 = s1;
                s1 = s1 + “ are immutable”;
                System.out.println(s1);
                System.out.println(s2);
        }
}
```

# Esercizio 6b *String* VS *StringBuilder*

Cosa stampa il seguente programma?

```
public class StringDemoB {
        public static void main(String[] args){
                String s1 = “Strings”;
                String s2 = s1;
                s1 = s1 + “ are immutable”;
                System.out.println(s1);
                System.out.println(s2);
        }
}
```

**Risposta:**

- Strings are immutable
- Strings

➢ s1 ed s2 puntano inizialmente allo stesso oggetto *String*

➢ il *concatenamento* di s1 con un’altra stringa genera un nuovo oggetto, al quale viene fatto puntare s1

➢ s2 continua a puntare all’oggetto precedente

# Esercizio 6b *String* VS *StringBuilder*

Quindi gli oggetti ***String*** sono sequenze di caratteri **non modificabili**.
Java fornisce un'alternativa per creare sequenze di caratteri **modificabili**: la classe ***StringBuilder***.

Replichiamo l'esercizio appena svolto utilizzando la classe *StringBuilder*.

# Esercizio 6b *String* VS *StringBuilder*

Cosa stampa il seguente programma?

```
public class StringDemoB {
    public static void main(String[] args){
        StringBuilder s3 =  new StringBuilder().append(“Stringbuilders”);
        StringBuilder s4 = s3;
        s3.append(“ are mutable”);
        System.out.println(s3);
        System.out.println(s4);
    }
}
```

# Esercizio 6b *String* VS *StringBuilder*

Cosa stampa il seguente programma?

```
public class StringDemoB {
    public static void main(String[] args){
        StringBuilder s3 =  new StringBuilder().append(“Stringbuilders”);
        StringBuilder s4 = s3;
        s3.append(“ are mutable”);
        System.out.println(s3);
        System.out.println(s4);
    }
}
```

**Risposta:**

- Stringbulders are mutable
- Stringbuilders are mutable

➢ s4 punta all’oggetto puntato da s3

➢ ogni modifica all’oggetto puntato da s3 è visibile anche da s4

# Esercizio 6c *StringWrapper*

Abbiamo visto che gli oggetti *String* sono sequenze di caratteri non modificabili, mentre gli oggetti *StringBuilder* sono sequenze di caratteri modificabili. Definire una classe wrapper per *String*, che emuli *StringBuilder*.

**Specifiche**:

- Deve esserci un attributo di tipo *String*, con getter e setter standard.
- Deve essere possibile il concatenamento con una seconda stringa.